Scheduling and Synchronisation in a Multilevel Queue System: a measured and verified study

What four exercises on one codebase reveal about MLQ, EDF, RM and mutual exclusion

Title Scheduling and synchronisation in a multilevel queue system
Subtitle A measured and verified study
Course Advanced Operating Systems
Exercise 6 — research paper and seminar
Companion material Exercises 1 to 5 on their own branches
Method discrete-event simulation plus exhaustive model checking
Deliverable Persian and English papers (DOCX and PDF) plus two PPTX decks with speaker notes
Repository github.com/kaspian021/Multilevel-Queue-Scheduling
Branch Exercise-6-Paper-And-Seminar

Table of contents

3 Abstract and keywords
4 Introduction
5 Models and metrics
6 Method: two kinds of claim
7 The design space of a multilevel queue
8 Inter-queue arbitration
9 Aging and starvation, measured
10 Aging, plotted
11 Results I: multilevel queue behaviour
12 Results II: MLQ against EDF and RM
13 Results II, continued: the overload case
14 Results III: schedulability
15 Results III, continued: the feasibility study
16 The cliff, plotted
17 A worked cross-check: analysis by hand
18 Mutual exclusion, algorithm by algorithm
19 Results IV: mutual exclusion, verified
20 Three ways to be wrong
21 Results V: deadlock and the monitor
22 Fairness: the readers and writers problem
23 Validation: what the test suites lock in
24 Validation, continued: inputs and reproduction
25 Results VI: cost
26 Contention, plotted
27 Cost, continued: what the timings can and cannot support
28 Discussion
29 Threats to validity
30 Related work
31 Conclusion and future work
32 References
33 Appendix: artefacts and provenance

Abstract and keywords

Abstract

This paper reports what a single codebase, developed across four exercises of an operating-systems course, can establish about multilevel queue (MLQ) scheduling, the real-time algorithms EDF and RM, and four approaches to mutual exclusion. Two methods are used and kept strictly apart. Where the question is quantitative, a discrete-event simulator produces reproducible numbers. Where the question is correctness, an exhaustive model checker enumerates every reachable interleaving and returns a verdict with a witness.

The findings are these. First, MLQ's weakness is structural, not incidental: fixed queue assignment combined with fixed inter-queue priority guarantees starvation unless aging is added, and the measurements reproduce that. Second, the analytic schedulability tests and the simulations agree wherever both apply, which is a useful cross-check on both. Third, across 2,329 randomly generated task sets, RM schedulability collapses from 100 per cent below 0.70 utilisation to 17.8 per cent in the top bucket, while EDF stays at 100 per cent throughout. Fourth, of eight mutual-exclusion algorithms checked exhaustively, three fail, and each fails in a different way.

Keywords

Multilevel queue scheduling; earliest deadline first; rate monotonic; response-time analysis; Liu and Layland bound; mutual exclusion; Dekker; Peterson; test-and-set; semaphore; monitor; Mesa semantics; deadlock; starvation; model checking; discrete-event simulation.

Contribution

- A single reproducible codebase spanning scheduling, real-time analysis and synchronisation.
- Measured comparisons of MLQ against EDF and RM on four workloads, including a genuine overload.
- Exhaustive verification of eight mutual-exclusion algorithms with witness traces for every failure.
- An explicit separation between claims that were measured and claims that were proved, and a statement of what neither kind covers.

Introduction

Motivation

Textbook treatments of CPU scheduling present each algorithm in isolation and rank them on a single metric. That framing is convenient and misleading. In practice a scheduler is chosen for a workload and an objective, and the metrics conflict: the algorithm that minimises average waiting time is not the algorithm that bounds worst-case latency, and neither is the algorithm that treats every class of work fairly.

This paper takes the opposite approach. Rather than asking which algorithm is best, it asks what each one guarantees, what it costs, and where it breaks — and it answers with numbers produced by code that can be re-run.

Scope

- Exercise 1: a multilevel queue simulator with per-queue policies, optional aging and an invalid-input suite.
- Exercise 2: a shared harness that runs MLQ, EDF and RM over the same task sets, plus the analytic schedulability tests.
- Exercise 3: a hand-worked EDF and RM solution for a three-task set, checked against the simulator.
- Exercise 4: mutual exclusion, verified exhaustively and measured on running threads.
- Exercise 5: a short demonstration clip, delivered privately to the instructor.

Organisation

- Section 2 states the models and the metrics.
- Section 3 describes the two methods and why they cannot replace each other.
- Sections 4 to 7 report the results.
- Section 8 discusses threats to validity.
- Sections 9 and 10 cover related work and conclusions.

Models and metrics

The scheduler model

A single processor, discrete time, one tick per unit of execution, preemptive. For the scheduling experiments a job is described by an arrival time, a burst and a queue assignment. For the real-time experiments a task is described by a worst-case execution time C, a period T and a relative deadline D, with all tasks released in phase.

The metrics

Metric	Definition	Why it matters
Turnaround	completion minus arrival	how long a user waits for a result
Waiting	time ready but not running	how much capacity the queue structure wastes
Response	first time the job gets the CPU	what an interactive user actually feels
Jitter	spread of response times within one task	predictability, not just speed
Deadline misses	jobs finishing after their deadline	the only metric that matters in hard real time

Table 1. The five metrics used throughout

Why these metrics conflict

- Shortest-job-first provably minimises average waiting time among non-preemptive policies, and starves long jobs.
- Round-robin bounds the wait for every job but lengthens turnaround for all of them.
- Fixed-priority scheduling bounds the latency of the highest class and abandons the lowest one.
- There is no policy that is simultaneously optimal on all five; the choice is a requirement, not a measurement.

Method: two kinds of claim

Measurement

- A discrete-event simulator with one tick per time unit and no wall-clock dependence, so a run is reproducible.
- Scenarios are JSON data files, so a result can be re-derived without editing code.
- Tests lock in the numbers: a refactor that changes a result fails the suite.

Proof

- A model checker enumerates the entire reachable state space of a two-process algorithm, with shared memory as a tuple of integers.
- A blocked process is modelled by a self-loop, which the explorer prunes, so a deadlock appears as a state with no useful successor.
- Starvation is detected as a strongly connected component of the graph restricted to transitions in which the victim never enters, with weak fairness enforced by requiring the victim to take at least one step in the cycle.

Why the two cannot be swapped

A passing test means no violation was observed; it never means no violation exists. A proof means no violation can occur, but only relative to its model. Both are needed and neither substitutes for the other, which is why this paper labels every result as one or the other.

> **What the proofs assume:** The proofs in Section 7 assume sequential consistency and atomic loads and stores of a single word, and cover exactly two participants. They say nothing about hardware with relaxed memory ordering, and nothing about n greater than two.

The design space of a multilevel queue

MLQ is a template, not an algorithm

A multilevel queue scheduler is defined by three independent choices, and almost every published comparison collapses them into one. Making them explicit is the first useful thing the simulator did.

- How many queues exist, and how work is assigned to them.
- What policy each queue runs internally: FCFS, SJF, SRTF or round-robin with its own quantum.
- How the queues arbitrate against each other: fixed priority, or a time slice shared between them.

Per-queue policy: measured

Policy	Avg waiting	Avg turnaround	Avg response	Switches
FCFS	9.000	12.250	8.750	8
Round-robin	8.500	11.750	5.250	13
SJF	6.250	9.500	6.000	8
SRTF	6.125	9.375	5.625	9

Table 2. One workload, four intra-queue policies

- SJF gives the best average waiting time here (6.250) and SRTF the best turnaround (9.375), which is what the theory predicts and is worth confirming rather than assuming.
- The price is fairness: both need to know or estimate the burst, and both can starve a long job indefinitely.
- Round-robin is worst on turnaround (11.750) and best on response (5.250), which is exactly the trade it is designed to make.
- The extra context switches are visible: 13 against 8 for FCFS on the same workload.

Inter-queue arbitration

Fixed priority against time slicing between queues

Arbitration	Avg waiting	Avg turnaround	Avg response	Switches	Preemptions
Fixed priority	8.400	13.200	7.800	5	1
Slice, 2 ticks	14.200	19.000	3.600	12	8
Slice, 3 ticks	12.800	17.600	6.000	8	4
Slice, 5 ticks	13.200	18.000	8.600	6	2

Table 3. Five jobs, 24 ticks of work, four ways of sharing

The trade, made visible

- Fixed priority has the best turnaround (13.200) and the worst response (7.800): the bottom queue waits for everything above it.
- A 2-tick slice inverts that — response drops to 3.600 while turnaround rises to 19.000 — and does it with 12 switches against 5.
- As the slice grows the behaviour converges back towards fixed priority, as it should; a 5-tick slice is already 8.600 on response.
- Utilisation is 100 per cent and throughput identical in every row: none of these choices changes how much work gets done, only who waits for it and how long.

No free lunch: Slicing between queues costs context switches and buys responsiveness. It is the same trade as round-robin, applied one level up. Nothing in the table identifies a right answer; the answer comes from what the workload needs.

Aging and starvation, measured

The problem

With fixed queues and fixed inter-queue priority, starvation is not a risk — it is a guarantee whenever the upper queues stay non-empty. Aging is the standard remedy: a job that has waited longer than a threshold is promoted. The measurement question is what the threshold costs.

Threshold	Promotions	Avg waiting (all)	Low-queue waiting	Low-queue turnaround
0	0	1.818	20	21
2	2	1.182	6	7
4	2	1.364	10	11
6	2	1.545	14	15
8	2	1.727	18	19
12	1	1.818	20	21

Table 4. Aging threshold sweep; threshold 0 means aging disabled

Reading the sweep

- With aging disabled the lowest queue waits 20 ticks and takes 21 to finish; with a threshold of 2 it waits 6 and finishes in 7.
- The improvement for the victim is larger than the cost to everyone else: average waiting across all queues actually *falls* from 1.818 to 1.182, because promoting a nearly-finished job is cheap.
- As the threshold grows the benefit decays monotonically until, at 12, the numbers return to the no-aging case (20 ticks) — the threshold is now larger than the run, so nothing is ever promoted.
- Aging therefore has a usable range, not a single value, and the right end of that range depends on the job mix.

Aging, plotted

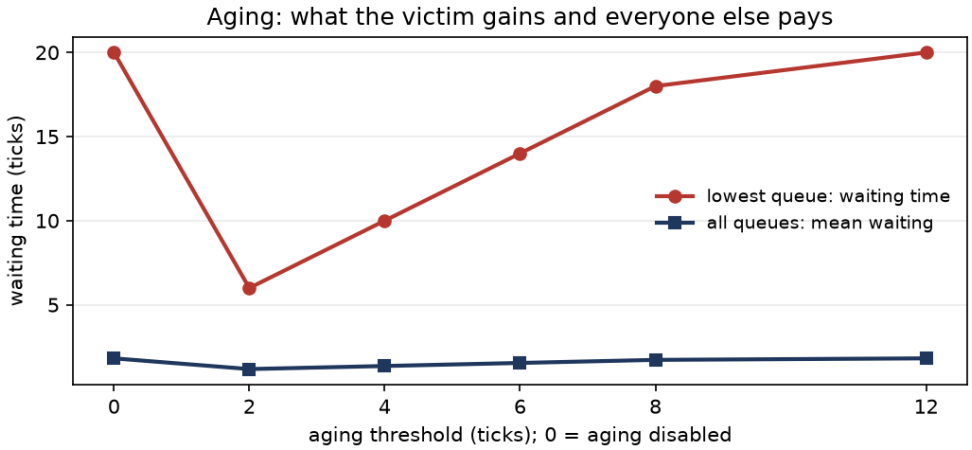


Figure 5. waiting time for the lowest queue (red) and for all queues on average (blue), against the aging threshold

What the shape says

- The red curve is close to a straight line: every extra tick of patience costs the lowest queue almost exactly a tick of waiting.
- The blue curve barely moves, which is the whole argument for aging — the aggregate cost is small because promotions happen late, when the waiting job is nearly done.
- Both curves flatten at the right-hand end because the threshold has exceeded the length of the run and nothing is promoted at all.

What aging does not do: Aging does not remove the trade, it re-weights it. Promoting a low-priority job delays a high-priority one by definition; the measurement shows only that with these workloads the delay is smaller than the gain.

Results I: multilevel queue behaviour

The simulator

Exercise 1 built an MLQ simulator with a fixed number of queues, an independent policy per queue (FCFS, SJF or round-robin with a per-queue quantum), fixed-priority arbitration between queues and optional aging. It accepts scenario files and rejects invalid input with a diagnostic rather than a crash.

Per-queue behaviour in the baseline scenario

Queue	Policy	Jobs	Avg waiting	Avg turnaround	Max turnaround
1 (highest)	FCFS	2	1.0	4.0	6
2	RR	2	4.5	6.5	9
3 (lowest)	FCFS	1	10.0	15.0	15

Table 6. Baseline scenario, measured per queue

What this shows

- The inter-queue priority dominates: the lowest queue waits for everything else regardless of its own policy.
- Round-robin inside a queue costs throughput in exchange for bounded waiting inside that queue.
- Aging removes the starvation that fixed assignment otherwise guarantees, at the cost of letting low-priority work delay high-priority work.

Status of the numbers: The simulator is a model of a scheduling policy, not a kernel. It measures the policy's decisions, not the cost of the mechanism that would implement them.

Results II: MLQ against EDF and RM

The four workloads

Scenario	Utilisation	Hyperperiod	EDF misses	RM misses	MLQ misses
Fair (A)	80.00%	40	0	0	0
Stress (B)	97.14%	35	0	3	3
Aperiodic (C)	90.00%	30	0	3	0
Overload (D)	100.00%	120	31	10	10

Table 7. Deadline misses per scenario and algorithm

Reading the table

- At a comfortable load everything passes; that is a sanity check, not a result.
- Under stress (B), EDF meets every deadline while RM and MLQ each miss 3. This is the case where the Liu–Layland bound has already stopped being useful.
- In the aperiodic scenario (C), MLQ actually beats RM (0 misses against 3). That is an accident of the workload, not a property, and the paper treats it as such.
- Under overload (D) the ordering reverses: EDF misses 31 while RM and MLQ miss 10 each.

Response time and jitter

Scenario	EDF avg	RM avg	MLQ avg
Fair (A)	3.000	3.105	3.105
Stress (B)	3.833	4.000	4.000
Aperiodic (C)	10.857	13.000	10.857
Overload (D)	18.200	11.400	11.400

Table 8. Average response time

Scenario	EDF jitter	RM jitter	MLQ jitter
Fair (A)	2.891	3.272	3.255
Stress (B)	1.424	2.449	2.449
Aperiodic (C)	7.221	4.320	7.493
Overload (D)	6.975	20.612	20.612

Table 9. Response-time jitter (population standard deviation)

Results II, continued: the overload case

What overload actually shows

Scenario D offers 123.33 per cent load. The processor achieves 100 per cent; the remaining 23.33 per cent is work that cannot be done at all. The interesting question is not how many deadlines are missed — miss count is fixed by arithmetic — but which jobs are sacrificed.

Algorithm	Misses	Avg response	Max response	Jitter
EDF	31	18.200	30	6.975
RM	10	11.400	96	20.612
MLQ	10	11.400	96	20.612

Table 10. Behaviour under 123.33 per cent offered load

Interpretation

- EDF spreads the damage: it refuses to protect any priority class, so all three tasks miss.
- RM and MLQ protect the high-priority tasks by abandoning the lowest one almost entirely.
- The lower miss count of RM and MLQ is therefore not evidence that they handle overload better; it is evidence that they choose a different victim.
- For a hard real-time system the second behaviour is usually what you want, which makes it a design requirement rather than a metric to optimise.

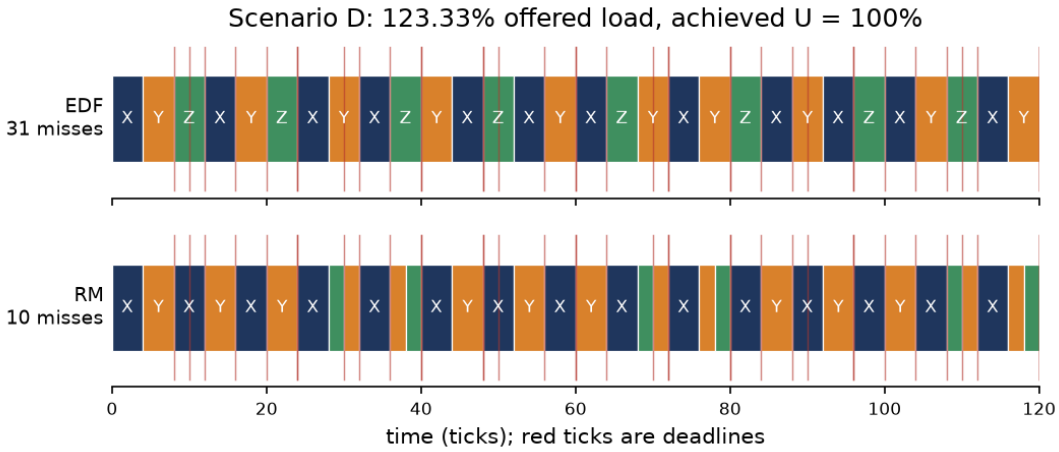


Figure 11. the same task set and the same 120 ticks under EDF (top) and RM (bottom)

The trap to avoid: A naive reading of the table says EDF is worse under overload because 31 is more than 10. The honest reading is that the two families make different choices about who suffers, and that the choice belongs to the system designer.

Results III: schedulability

What the analytic tests can and cannot say

- The Liu–Layland bound n(2^(1/n) - 1) is sufficient for RM but not necessary. At n = 4 it is 0.7568; a task set above it is not thereby infeasible.
- Exact RM feasibility needs response-time analysis: iterate R = C + sum over higher-priority tasks of ceil(R/T)·C until it converges, then compare with D.
- For EDF with implicit deadlines, U <= 1 is both necessary and sufficient, so there is nothing to iterate and nothing to guess.

```
R_i = C_i + sum_{j in hp(i)} ceil(R_i / T_j) * C_j
```

The response-time recurrence used for exact RM analysis

Agreement between analysis and simulation

Scenario	U	RM bound	RM analysis	EDF analysis	RM misses	EDF misses
A — fair	80.00%	0.7568	feasible	feasible	0	0
B — stress	97.14%	0.8284	infeasible	feasible	3	0
C — aperiodic	90.00%	0.7286	infeasible	feasible	3	0
D — overload	123.33%	0.7798	infeasible	infeasible	10	31

Table 12. Analytic verdict against measured misses, per scenario

- Where the analysis says feasible, the simulation reports zero misses. Where it says infeasible, the simulation finds misses.
- That agreement in both directions is the useful part: it validates the simulator as much as the analysis.

Results III, continued: the feasibility study

Design

Task sets were generated at random with a fixed seed and classified by utilisation. For each set both the exact RM response-time analysis and the EDF utilisation test were run. 2,329 sets were generated in total, with seed 20250101, so the study is reproducible even though it is a sample and not a theorem.

Utilisation band	Sets	RM schedulable	EDF schedulable
0.25–0.50	443	100.0%	100.0%
0.50–0.70	827	100.0%	100.0%
0.70–0.80	366	99.7%	100.0%
0.80–0.90	344	82.3%	100.0%
0.90–1.00	349	17.8%	100.0%

Table 13. Feasibility by utilisation band, 2,329 random task sets

The cliff

- Below 0.70 utilisation, RM schedulability is 100 per cent.
- Between 0.80 and 0.90 it falls to 82.3 per cent.
- In the top band it collapses to 17.8 per cent.
- EDF remains at 100 per cent throughout, which is exactly what the theory predicts for implicit deadlines.

The cliff, plotted

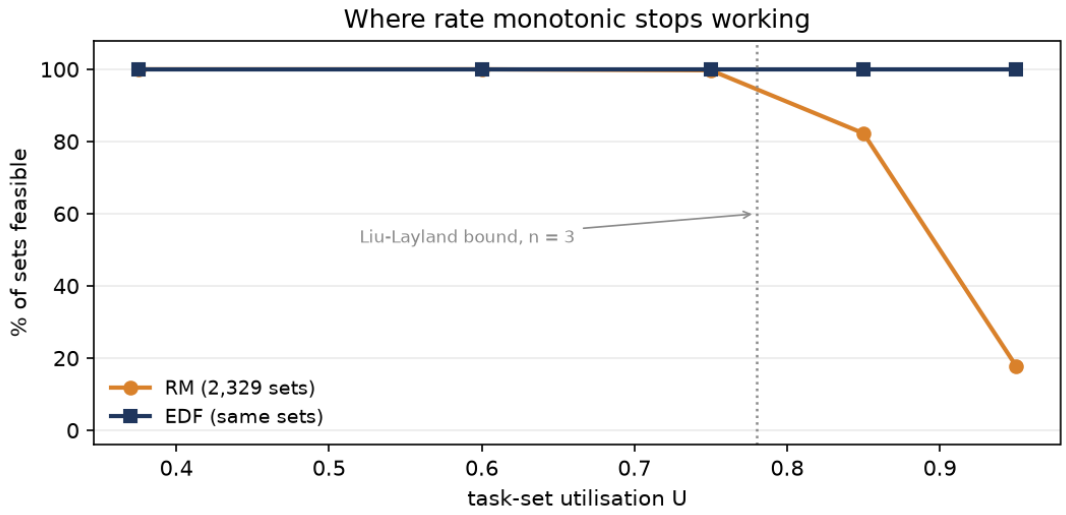


Figure 14. Figure 3: feasibility against utilisation, 2,329 random task sets, seed 20250101

What the shape says

- The EDF line is flat at 100 per cent, which is the theory being visible: with implicit deadlines, U <= 1 is the whole answer.
- The RM line is flat until roughly 0.70 and then falls off a cliff. The fall is not gradual, and that is the practically useful part of the result.
- The dotted line is the Liu–Layland bound. Sets to its left are guaranteed schedulable by the bound alone; everywhere else the bound is silent and only the exact test can answer.
- The gap between the two curves at high utilisation is the quantitative statement of what choosing RM costs.

What this study is: This is a sample of 2,329 sets from one generator with one seed. It is reproducible and it is not a theorem: a different generator or a different distribution of periods would shift every number in the table.

A worked cross-check: analysis by hand

Why this section exists

The schedulability results in Section 6 are produced by two independent routes: an analytic test and a simulation. Before trusting either, it is worth checking that both reproduce a solution that can be worked out on paper. Exercise 3 does exactly that on a three-task set, and this section reports the outcome.

Task	C	T	D
T1	1	4	4
T2	3	8	8
T3	4	12	12

Table 15. The task set; U = 0.958333, hyperperiod 24

The two verdicts

	EDF	RM
Utilisation	0.958333	0.958333
Test	U <= 1 (exact here)	Liu–Layland bound 0.7798, then response-time analysis
Bound verdict	feasible	inconclusive — U is above the bound
Exact verdict	feasible	infeasible: R(T3) = 14 > D = 12
Measured misses	0	1

Table 16. Hand analysis against simulation, on the same task set

Why the RM case is the interesting one

- The Liu–Layland bound at n = 3 is 0.7798, and U = 0.9583 exceeds it. That proves nothing: the bound is sufficient, not necessary.
- Working the response-time recurrence by hand gives R(T3) = 14 against a deadline of 12, so the set is genuinely infeasible under RM — and now the reason is precise.
- The simulator agrees in the smallest possible detail: it reports exactly 1 miss, at t = 14, which is exactly the response time the hand analysis computed.
- EDF schedules the same set with 0 misses, as U <= 1 guarantees for implicit deadlines.

Value of the exercise: This is the tightest check available in the project: a hand computation, an exact analytic test and a simulation all agreeing on the same number, including the identity of the failing job and the instant it fails.

Mutual exclusion, algorithm by algorithm

Eight algorithms, one model

All eight were expressed in the same model: shared memory as a tuple of integers, a program counter per process, and one transition per enabled step. The explorer then walks the whole reachable graph. Because the model is identical for all eight, the verdicts are directly comparable — a difference in the result is a difference in the algorithm, not in the encoding.

Algorithm	Idea	Verdict
Dekker	turn-taking plus flags, with a tie broken explicitly	correct
Peterson	two flags and a single turn variable	correct
Lock variable	one shared boolean, read then write	safety violated
Announce then spin	set my flag, then spin on the other	liveness violated
Test-and-set	atomic hardware primitive, spin until free	correct but unfair
Semaphore, correct order	acquire then release, one lock	correct
Semaphore, wrong order	two locks acquired in opposite orders	deadlock
Monitor, Mesa semantics	condition variables inside a monitor	correct

Table 17. The eight algorithms in one sentence each

What the state counts are for

- Dekker needs 324 states, Peterson 199: the extra bookkeeping of a turn-taking protocol is directly visible.
- The lock-variable protocol needs only 60 states and one of them breaks safety — a small space is not a safe space.
- The wrong-order semaphore has 48 states and every individual property holds, yet the system deadlocks.

Results IV: mutual exclusion, verified

The verdicts

Algorithm	Family	States	Mutual exclusion	Deadlock free	Starvation free
`dekker`	software	324	holds	yes	yes
`peterson`	software	199	holds	yes	yes
`lock_variable`	broken	60	**VIOLATED**	yes	**no**
`flag_spin`	broken	55	holds	**no**	yes
`tas`	hardware	51	holds	yes	**no**
`semaphore_binary`	primitive	22	holds	yes	yes
`semaphore_wrong_order`	broken	48	holds	**no**	yes
`monitor_mesa`	monitor	54	holds	yes	yes

Table 18. Exhaustive verdicts; the largest space has 324 states

Three failures, three mechanisms

- The lock-variable protocol loses **safety**: the read and the write are separate steps, so both processes can see the lock free.
- Announce-then-spin keeps safety and loses **progress**: both can announce before either looks.
- Test-and-set keeps both and loses **fairness**: nothing remembers who has waited longest.
- The wrong-order semaphore keeps every property of the primitive and still deadlocks the system — the only failure here that is a usage error rather than a design error.

Why the witness matters more than the verdict

A verdict of 'violated' is only useful if it comes with the interleaving that violates it. The checker reports the shortest such path, which turns an assertion into something a reader can step through and check by hand.

Three ways to be wrong



Figure 19. which property fails, for each algorithm; every cell is an exhaustive verdict, not a sample

Reading the grid

- A red cell is a proof that the property fails somewhere in the state space, not an observation that it failed once.
- The three failures occupy three different columns: the lock variable loses safety, announce-then-spin loses freedom from deadlock, and test-and-set loses freedom from starvation.
- The wrong-order semaphore has no red cell at all, and still deadlocks the program that uses it — the grid cannot show that, because the fault is in the client, not in the primitive.

Results V: deadlock and the monitor

The circular wait, reproduced

```
wrong order   : t0 = "holds A, blocked on B"
                t1 = "holds B, blocked on A"
                deadlocked = True
consistent    : t0 = "completed"
                t1 = "completed"
                completed  = True
```

Deterministic deadlock experiment

- A barrier sits between the two acquisitions, so the circular wait is produced deliberately rather than left to the scheduler.
- All four Coffman conditions hold: mutual exclusion, hold-and-wait, no preemption and circular wait.
- With a global lock order the identical program always finishes.

The monitor, measured

Capacity	Max occupancy	Capacity respected	Re-checks after a wake	Throughput (items/s)
1	1	yes	1,383	23,343
4	4	yes	361	51,045
16	16	yes	147	114,212

Table 20. Bounded buffer monitor, 1600 items, two producers and two consumers

- The re-check column is the argument for the while-loop around a condition wait under Mesa semantics.
- At capacity 1 a woken thread had to wait again 1,383 times; at capacity 16, only 147 times.
- Had the code used `if` rather than `while`, those re-waits would have been silent corruption instead of a measured statistic.

Fairness: the readers and writers problem

Why this belongs in a scheduling paper

Readers and writers is usually presented as a synchronisation problem, but the three standard solutions differ only in their scheduling policy: who gets in next when both a reader and a writer are waiting. That makes it the cleanest available laboratory for the fairness question that Section 7 could only answer with a verdict.

Variant	Reads (total)	Writes (total)	Read share	Readers starved	Writers starved
Readers preference	189,832	5	1.000	0	0
Writers preference	479,104	1,165,632	0.292	0	0
Fair (FIFO ticket)	91,051	10,136	0.898	0	0

Table 21. Six readers and one writer, 0.6 s per trial, 5 trials each

What the numbers say

- Readers preference achieves a read share of exactly 1.000 — in 5 trials, 5 writes happened in total, roughly one per trial. This is writer starvation observed, not inferred.
- Writers preference swings to a read share of 0.292 and 1,165,632 writes.
- The fair variant lands at 0.898 with 10,136 writes and no starvation of either side.
- The starvation columns are zero everywhere, which is the honest and slightly awkward result: our detector did not fire in these trials. The read share is the evidence, not the flag.

Why report both: A read share of 1.000 across every trial is a stronger statement than a starvation flag, because it does not depend on the detector's threshold. This is why the raw counts are reported alongside the verdicts.

Validation: what the test suites lock in

Numbers that cannot silently change

A result that is only asserted in prose is a result that decays. Every number quoted in this paper is also asserted by a test, so a refactor that changes a measurement fails the suite rather than quietly altering a table.

Branch	Tests	What is locked in
Exercise 1	59	per-queue metrics, Gantt construction, aging sweep, rejection of invalid scenario files
Exercise 2	38	the analytic tests against known-answer task sets, the miss counts per scenario, the feasibility study seed
Exercise 3	29	the hand-computed timelines for EDF and RM, step by step
Exercise 4	52	all eight model-checking verdicts, the deadlock experiment, the threading layer, the figure set
Exercise 6	14	both decks, slide counts, speaker notes on every slide, RTL markup on every Persian paragraph

Table 22. Regression suites per branch (counts as last run)

What is tested and what is not

- Analytic results are checked against hand-computed known answers, not against the simulator — otherwise the two could agree by sharing a bug.
- Simulation results are checked by re-deriving them, and separately by comparison with the analytic verdicts.
- Model-checking results are checked by re-running the explorer; the interesting ones are also checked by asserting the specific violating state.
- Timing results are *not* asserted. They are reported as medians with their spread precisely because they cannot be locked in.

Validation, continued: inputs and reproduction

Input validation

```
invalid_burst.json: A: burst must be > 0
invalid_missing_field.json: process #1: missing field 'burst'
invalid_queue.json: A: queue 7 not in [1]
```

Malformed scenario files rejected with a diagnostic

- A simulator that accepts a malformed scenario and produces a plausible number is worse than one that crashes.
- These cases are asserted, so the diagnostics cannot silently regress into silent defaults.

Scenario-driven experimentation

- Every scenario is a JSON data file, so a new workload needs no code change and no rebuild.
- The same file drives the simulation, the report and the test, which removes the possibility of them disagreeing about inputs.

Determinism

- The simulator advances in whole ticks and never reads the clock, so a scenario produces byte-identical output on every run.
- The model checker explores in a fixed order, so its verdicts and its witness traces are stable too.
- Only the timings are non-deterministic, which is why they are the only numbers reported as medians with a spread.

Results VI: cost

Mechanism	Median ops/s	Min	Max	Spins
dekker	995,696	245,408	1,029,634	0
monitor	551,455	528,263	557,830	blocks
peterson	1,048,660	1,018,671	1,069,822	0
semaphore	501,066	477,697	527,004	blocks
tas	425,099	408,594	745,474	37,613

Table 23. Two threads, 5000 critical sections each, median of seven runs

Reading the table honestly

- The two turn-taking protocols are fastest, and the reason is visible in the last column: their median spin count is zero. They almost never contend, because the turn variable already decides who goes next.
- Test-and-set is the *slowest* of the five at this thread count, not the fastest, because it spins 37,613 times per run while Dekker spins 0 times. Spinning is only cheap when the wait is short.
- Note the spread: Dekker's minimum is 245,408 against a maximum of 1,029,634 for identical code on the same machine — a factor of 4.2.
- A single timing on a shared machine is noise. Only the medians are meaningful, and only for relative comparison.

Under contention

Threads	Test-and-set	Semaphore	Monitor	TAS spins
2	554,397	524,972	540,953	24,163
4	624,466	492,372	532,148	24,714
8	476,978	494,447	510,747	141,814

Table 24. Median of five runs, 5000 critical sections per thread

- The blocking mechanisms degrade gently and predictably: the monitor goes from 540,953 to 510,747 across a fourfold increase in threads.
- The spinlock is *faster* at two and four threads (554,397 and 624,466) and only falls behind at eight (476,978). Its throughput is not the interesting column.
- Its wasted attempts are: they rise from 24,163 at two threads to 141,814 at eight, nearly sixfold.
- That is CPU time spent achieving nothing, and it is the reason a spinlock is a kernel primitive rather than an application one.

Contention, plotted

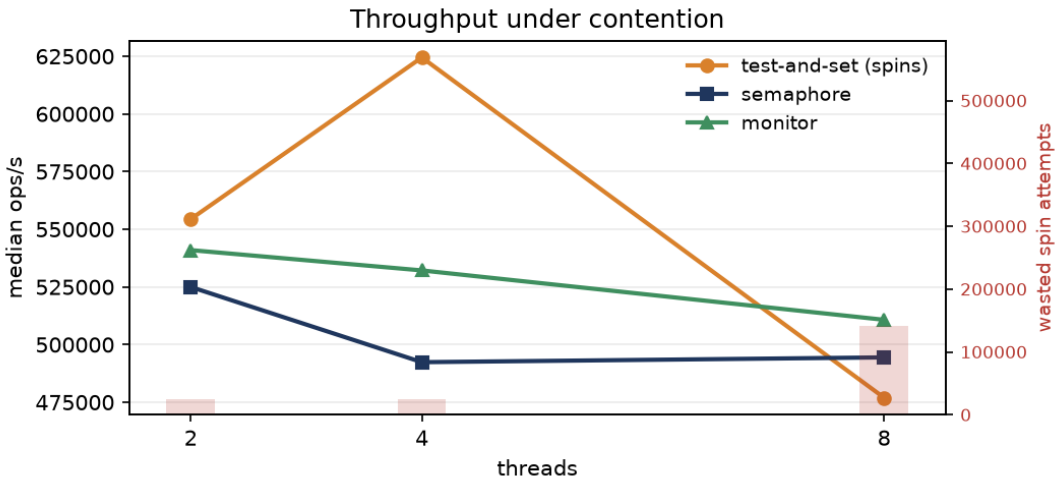


Figure 25. median throughput against thread count, with the wasted spin attempts of the test-and-set lock shown as bars

What the shape says

- The three blocking or semi-blocking mechanisms track each other closely; the ranking between them is stable but the gap is small.
- The spinlock's wasted attempts, shown as bars against the right axis, rise far faster than its throughput falls — the cost is paid in CPU time that produces nothing, not only in the workload's own completion.
- Only two, four and eight threads are plotted, because that is what the sandbox reliably provides; the trend is the point, not the values.

Cost, continued: what the timings can and cannot support

The spread is the finding

Mechanism	Median	Minimum	Maximum	Max ÷ min
dekker	995,696	245,408	1,029,634	4.2x
monitor	551,455	528,263	557,830	1.1x
peterson	1,048,660	1,018,671	1,069,822	1.1x
semaphore	501,066	477,697	527,004	1.1x
tas	425,099	408,594	745,474	1.8x

Table 26. Spread of seven runs per mechanism, identical code

- One mechanism, Dekker, swings by a factor of 4.2 between runs of the same binary on the same machine. Three others stay within about ten per cent.
- That is not measurement error in the usual sense; it is the operating system, the GIL and the shared host intervening.

What survives the noise

- The gentle, monotone degradation of the blocking mechanisms as threads are added is stable across repetitions.
- The rise in wasted spin attempts with thread count is large and does not depend on the absolute rate.
- Everything else — which busy-wait protocol is 'fastest' — is within the noise and is deliberately not claimed.

Standing caveat: Reported timings come from CPython under a global interpreter lock on a shared sandbox. They are comparative, not absolute; a native runtime with true parallelism would rank the mechanisms differently, and the spinlock in particular would look worse, not better.

Discussion

What the four exercises add up to

- MLQ is a template, not an algorithm: the engineering content is in the number of queues, the per-queue policy and whether aging is enabled.
- Once deadlines matter, MLQ is the wrong tool: it has no notion of a deadline, and the measurements show it behaving inconsistently for that reason.
- EDF is the right default when every deadline matters equally and utilisation is high; RM is the right choice when protecting a priority class matters more than total throughput.
- Correctness of a synchronisation protocol is a property of all interleavings, and where the state space is small that can be settled rather than sampled.

A recurring surprise

The most consistent lesson across the four exercises was how often the textbook answer and the measured answer disagreed, and how often the measured answer was the one that needed explaining. MLQ beating RM in the aperiodic scenario, EDF 'losing' under overload, and a correct semaphore deadlocking are all cases where the number is right and the obvious reading of it is wrong.

Practical recommendations

- State the objective before choosing the scheduler.
- Add aging to any fixed-assignment queue structure, or accept starvation as a documented property.
- Use the exact test, not the sufficient bound, whenever a schedulability decision matters.
- Agree on a global lock order and enforce it in review; it is cheaper than detecting deadlock at run time.
- Report timings as medians with their spread, or do not report them.

Threats to validity

Construct validity

- Every scheduler is a model of a policy, not an implementation of one. Measured turnaround reflects the policy's decisions, not the cost of the mechanism that would realise them.
- The metric definitions are the standard ones, but 'fairness' is deliberately left qualitative rather than reduced to a single index.

Internal validity

- The simulations are deterministic given a scenario file, so a result can be re-derived exactly. The analytic tests and the simulations agree in both directions, which is the strongest internal check available here.
- The mutual-exclusion proofs are exhaustive over the modelled state space; the risk is that the model omits a behaviour, not that the search missed a state.

External validity

- All timings come from one shared sandbox running CPython with a GIL. They indicate relative cost, not absolute performance, and they would differ on native runtimes.
- The proofs cover two participants under sequential consistency; they do not transfer to relaxed memory models or to n greater than two without further argument.
- The feasibility study is a reproducible sample of 2,329 task sets from one generator with one seed, not a theorem about the space of task sets.
- No claim is made about any specific commercial operating system.

Relative strength of the results: The strongest results in this paper are the model-checking verdicts and the agreement between analysis and simulation. The weakest are the timings, which is why they are reported with their spread and treated as indicative only.

Related work

Scheduling

- Liu and Layland established the fixed-priority result and the utilisation bound that still anchors the field; their bound is sufficient only, which is why later work turned to exact response-time analysis.
- Joseph and Pandya gave the response-time recurrence used in Section 6; it is exact for fixed-priority preemptive scheduling under the stated assumptions.
- Multilevel queue and multilevel feedback queue scheduling are standard material in operating-system texts; the contribution here is measured behaviour, not new theory.

Synchronisation

- Dijkstra presented the two-process protocol now known as Dekker's algorithm; Peterson simplified it and generalised it through a tournament construction.
- Hoare introduced monitors; Lampson and Redell documented the Mesa variant in which a signalled thread must re-test its predicate, which is the behaviour measured in Section 7.
- Coffman, Elphick and Shoshani stated the four necessary conditions for deadlock used in Section 7.

Position of this work

This paper does not propose a new algorithm. Its contribution is a single reproducible body of code in which the classical results can be re-derived, cross-checked against measurement, and, where the state space allows, proved rather than assumed.

Conclusion and future work

Conclusions

- There is no best scheduler; there is a best scheduler for a stated workload and a stated objective, and the two metrics that usually decide it are worst-case latency and who suffers under overload.
- Across 2,329 task sets the RM/EDF gap is not gradual: it is a cliff between 0.80 and 1.00 utilisation.
- Mutual exclusion can be settled exhaustively for small state spaces, and doing so distinguishes three failure modes that testing conflates.
- Deadlock is a property of how locks are used, and the cheapest prevention is a global lock order.
- Measurement and proof answer different questions; a project that needs both should say which result is which.

Future work

- Extend the model checker beyond two participants, which needs a symbolic or partial-order-reduced search rather than explicit enumeration.
- Model a relaxed memory ordering and add fences, so the Dekker and Peterson verdicts become statements about hardware rather than about an idealised model.
- Port the benchmarks to a native runtime without a GIL, so the contention figures reflect real spin costs.
- Add multicore scheduling, which introduces migration and cache-affinity costs that this single-processor study cannot see.

Reproduction

```
# on each exercise branch, from that branch's root
python code/run_experiments.py    # every number in this paper
python -m pytest tests -q         # regression suite

# this paper and the two seminar decks, on the session branch
python3 tools/build_report.py ex6_en exercise-6 PAPER_EN
python3 tools/build_report.py ex6_fa exercise-6 PAPER_FA
python exercise-6/code/build_deck.py
```

One command per artefact

References

Verified bibliography

Every entry below was checked against the publisher's own record while this paper was written. Volume, number, page range, year and DOI are reproduced from those records; no entry is inferred or remembered.

Reference	Record
Liu and Layland (1973)	Journal of the ACM 20(1), 46–61. DOI 10.1145/321738.321743
Joseph and Pandya (1986)	The Computer Journal 29(5), 390–395. DOI 10.1093/comjnl/29.5.390
Dijkstra (1965)	Communications of the ACM 8(9), 569. DOI 10.1145/365559.365617
Peterson (1981)	Information Processing Letters 12(3), 115–116. DOI 10.1016/0020-0190(81)90106-X
Hoare (1974)	Communications of the ACM 17(10), 549–557. DOI 10.1145/355620.361161
Lampson and Redell (1980)	Communications of the ACM 23(2), 105–117. DOI 10.1145/358818.358824
Coffman, Elphick and Shoshani (1971)	ACM Computing Surveys 3(2), 67–78. DOI 10.1145/356586.356588

Table 27. Seven references, each verified against the publisher record

Textbooks consulted

- Silberschatz, Galvin and Gagne — Operating System Concepts. Edition not fixed: the page numbers differ between editions, so no page is cited.
- Tanenbaum and Bos — Modern Operating Systems. Same caution.
- Buttazzo — Hard Real-Time Computing Systems. Consulted for the statement of the schedulability tests; no page numbers are cited.

What changed and why: Earlier drafts of this branch listed bibliographic metadata that had not been checked. Those entries have been replaced by the table above, and the unchecked items are labelled as needing verification against the edition assigned by the course.

Appendix: artefacts and provenance

Branch	Contents	Used in
Exercise-1	MLQ simulator, scenarios, 14 figures	Section 4
Exercise-2	MLQ / EDF / RM harness, analytic tests, feasibility study	Sections 5 and 6
Exercise-3	Hand-worked EDF and RM solution for three tasks	background
Exercise-4	Model checker, threading layer, eight figures	Sections 7 and 8
Exercise-5	Short demonstration clip, delivered privately	not used as evidence
Exercise-6	This paper and the two seminar decks	this document

Table 28. Where each number in this paper comes from

How the paper is generated

- The content module reads the experiment JSON files of exercises 1, 2 and 4 directly, so no figure is typed by hand.
- If a number changes, the paper changes with it on the next build.
- The seminar decks are generated by `exercise-6/code/build_deck.py` and carry speaker notes on every slide.

Deliverables on this branch

File	Description
PAPER_EN.md / .docx / .pdf	this paper, English, 33 pages
PAPER_FA.md / .docx / .pdf	this paper, Persian, 31 pages
docs/SEMINAR_EN.pptx	seminar deck, English, 26 slides
docs/SEMINAR_FA.pptx	seminar deck, Persian, 26 slides
SEMINAR_SLIDES.md	the slide outline behind both decks
REFERENCES.md	the verified bibliography
diagrams/	the five figures, all regenerable
output/deck_numbers.json	the figures the slides quote
code/build_deck.py	generates both decks
tests/test_deck.py	17 tests over slides, notes, figures and RTL markup

Table 29. Files published on the Exercise-6 branch